Queueing and Team Formation with Role Constraints

OPEN-SOURCE IMPLEMENTATION · MIT LICENSE

https://github.com/e-fei/moba-role-matchmaking

Abstract

Forming a five-player MOBA team requires one player in each role, but close skill matches may require longer waits. This report studies a policy that scans players in arrival order and widens the acceptable score band as they wait. For the first player whose band contains all five roles, a 32-state dynamic program minimizes the mean absolute score difference within the team. On five balanced-role simulations, the default policy matches all 5,000 arrivals, reduces mean score gap from 331.86 to 104.69 points, and raises mean wait from 6.72 to 9.12 seconds relative to role-wise FIFO. When support players are scarce, every policy reaches only 50% completion; the unmatched queue then changes how waiting-time results should be interpreted. An optional age reward preserves the 32-state optimizer and admits a bound on local score-quality loss. The scheduling policy remains heuristic.

1. Introduction

Consider an old request from a top player and a queue containing many mid and ADC players but no support. Neither a wider score band nor a more elaborate optimizer can form a team. Once a support arrives, however, selecting the oldest player in each role may produce a large skill gap. The assignment therefore involves two decisions: which waiting player to serve next, and which compatible teammates to select.

A strict score threshold makes team quality easy to control but can leave requests stranded. Ignoring scores removes that restriction at the cost of uneven teams. The policy used here starts with a narrow interval and relaxes it with age. It serves the oldest *feasible* request, rather than stopping behind the oldest request when a required role is unavailable.

Rating estimation is treated as an upstream task. TrueSkill, for example, infers player skill from game outcomes (Herbrich et al., 2006). Here the ranking score is supplied as input; the task is to assemble one team from existing scores. No opposing-team match or outcome prediction is required.

2. Problem and Queueing Policy

A player is represented by $p = (\mathrm{id}, r, s, a)$, where r is one fixed role, s is the score, and a is the request time. A valid team T contains exactly one top, jungle, mid, ADC, and support player. For a team formed at time t, define

$$W(T, t) = \frac{1}{5} \sum_{i \in T} (t - a_i), \tag{1}$$

$$G(T) = \frac{1}{10} \sum_{i<j;\, i,j \in T} |s_i - s_j|. \tag{2}$$

Equation (2) interprets the assignment's score gap as the mean over the ten unordered player pairs. The two objectives do not specify a unique optimum: waiting for future arrivals can improve G while increasing W. The widening rate below is the explicit tradeoff parameter.

2.1. Age-Dependent Eligibility

Scan the queue in arrival order. For a candidate anchor q, use the integer band

$$b_q(t) = \mathrm{floor}(\min\{B, b_0 + g(t - a_q)\}). \tag{3}$$

A teammate p is eligible if $|s_p - s_q| \le b_q(t)$. If any role is missing, move to the next anchor. For the first feasible anchor, minimize G among eligible teams that include it, remove those five players, and repeat.

The default parameters are $b_0 = 50$, $g = 10$ points/s, and $B = 10^6$. Scores lie in $[0, 10^6]$. Equal-time anchors keep insertion order. Equal-gap teams are ordered by their sorted (request time, ID) lists, with earlier times and then smaller IDs preferred.

2.2. Maintaining the Queue

An arrival-ordered vector preserves waiting priority. For each team selection, a score-sorted copy and five prefix counters allow binary-search interval lookup and role-availability checks. A global five-bit role mask avoids sorting if a role is absent from the entire queue.

The band belongs to the anchor, not to each teammate: two selected scores can differ by twice its width. Matching is reconsidered after arrivals and on a timer; the simulator uses one-second ticks.

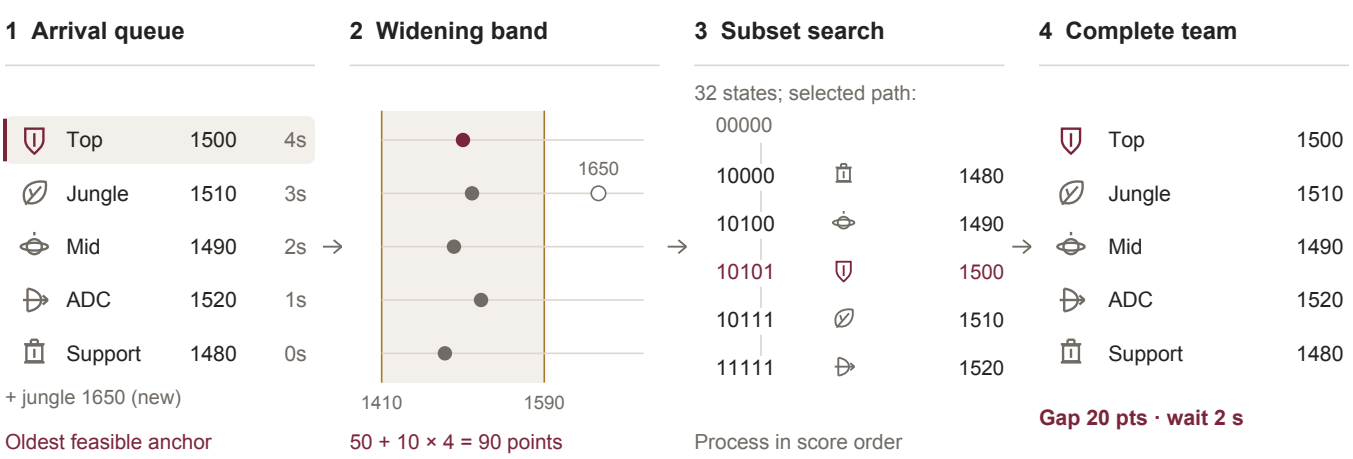


Figure 1. From queue to team at time 4 s. Role emblems identify the same players across stages; the top anchor is highlighted. The band also rejects a jungle candidate at 1650. The search considers all 32 role subsets; only its selected path is drawn. Bits, from right to left, represent top, jungle, mid, ADC, and support. Selected requests are removed together.

3. Exact Team Selection

Figure 1 follows one request through the policy. Selecting the nearest player to the anchor in each role need not minimize the pairwise team gap. A subset dynamic program evaluates the choices jointly without enumerating every combination.

3.1. An Additive Form of the Objective

Write the selected scores in ascending order as $x_0 \le \ldots \le x_4$. A score at position k appears positively in k differences and negatively in $4 - k$ differences. Hence

$$10G(T) = \sum_{k=0}^{4} (2k - 4)x_k \\ = -4x_0 - 2x_1 + 2x_3 + 4x_4. \tag{4}$$

Each role corresponds to one bit. For a score-sorted prefix, $D[M]$ records the least weighted sum among selections with role mask M. Only the empty mask is initially reachable. If k roles have been chosen, selecting score s adds $(2k - 4)s$. Excluding every other player with the anchor's role forces the anchor into any complete team.

There are $2^5 = 32$ role subsets, regardless of queue size. Intermediate costs may be negative; only the completed expression represents a sum of absolute differences.

Algorithm 1 Minimum-gap team for an anchor

Input: anchor q, eligible players sorted by (score, ID).
Output: a team containing q.

1 Set $D[0]$ to the empty selection; all other states are unreachable.
2 **for** each eligible player p **do**
3 Skip p if it has q's role but is not q.
4 Let u be the bit for p's role.
5 **for** masks $M = 31,\ldots,0$ **do**
6 Skip unreachable states and masks containing u.
7 Set k = popcount(M).
8 Extend $D[M]$ by p, adding $(2k - 4)s_p$.
9 Improve $D[M \mid u]$ by cost, then the age/ID key.
10 **return** the five players in $D[31]$.

Here | denotes bitwise OR. Descending masks prevent a state updated for the current player from being reused in the same pass. The final cost is divided by ten.

The same search will support the waiting-aware extension in Section 4: only the incremental cost in line 8 changes. Eligibility, anchor priority, and the number of states stay fixed.

3.2. Correctness

Proposition 1. *For a fixed feasible anchor and band, Algorithm 1 returns a valid team with minimum mean pairwise score gap, using the stated tie rule.*

Proof. Every transition adds one unused role, so mask 31 contains five distinct roles. All candidates satisfy the band constraint, and the anchor is the only available player for its role.

Induct on the processed prefix. Partial selections with the same mask have the same size and the same remaining candidates. By Eq. (4), every common completion adds the same future cost to both. A more expensive partial selection can therefore be discarded. Every legal team appears as a score-ordered subsequence of skip/select decisions, so the cheapest complete state is optimal.

For equal costs, inserting a common future player into two sorted age/ID keys preserves their lexicographic order. The tie-breaking state is therefore safe to retain. □

Why Nearest-Neighbor Selection Fails

Fix a top anchor at 1500 and band 50. Mid, ADC, and support each have score 1550. Jungle offers two choices:

Jungle choice	Anchor distance	Team gap
1460 (nearest)	40	46
1550 (joint optimum)	50	20

Choosing 1460 is closer to the anchor but farther from three teammates. The pairwise objective therefore favors 1550.

3.3. Cost and Scope

Table 1. Cost of one team selection; $m \le n$.

Operation	Worst-case time
Sort and role prefix counts	$O(n \log n)$
Find a feasible anchor	$O(n \log n)$
DP over m candidates	$O(32m)$
Remove five players	$O(n)$

One selection takes $O(n \log n + 32n)$ time and $O(n)$ auxiliary space. Rebuilding the index after each team gives an $O(n^2 \log n)$ batch bound. The proof is local: greedy removals need not minimize total gap or maximize throughput over later teams.

4. Serving Older Teammates

Anchor priority does not favor older teammates when a younger candidate has a smaller gap. An optional age reward uses $C = 120$ s and $\lambda \ge 0$:

$$J_\lambda(T) = G(T) - \lambda R(T),$$
$$R(T) = \frac{1}{5} \sum_{i \in T} \min(t - a_i, C). \tag{5}$$

Keep the 32 states and replace line 8 by this cost, where k is the number of selected roles:

$$\Delta = (2k - 4)s_i - 2\lambda \min(t - a_i, C). \tag{6}$$

This minimizes $10J_\lambda$ exactly. The weight λ has units of points/s; tested values use integer-scaled costs for exact ties. Setting $\lambda = 0$ recovers the baseline.

4.1. An Interpretable Quality Budget

Proposition 2. *For the same anchor and candidate pool, let T_0 minimize G and T_λ minimize J_λ. Then*

$$0 \le G(T_\lambda) - G(T_0)$$
$$\le \lambda[R(T_\lambda) - R(T_0)] \le 4\lambda C/5. \tag{7}$$

Proof. Optimality gives $J_\lambda(T_\lambda) \le J_\lambda(T_0)$. Rearranging yields the first upper bound. The shared anchor cancels in the age difference; each of the other four ages lies in $[0, C]$. □

Choosing $\lambda = 5\varepsilon/(4C)$ limits the local gap penalty to ε points. For $\lambda = 0.1$, the bound is 9.6 points. This is a sufficient bound for one selection, not a guarantee about the final online schedule.

4.2. Exploratory Sweep

Table 2. Scarce-support sweep on five streams (Section 5). Wait is matched-player mean; pending p95 is cutoff age. Completion is 50% throughout.

λ	Gap (pts)	Wait (s)	Pending p95 (s)
0	113.48	34.57	256
0.1	118.02	40.45	244
0.25	109.91	42.24	238
0.5	115.32	42.97	235
1	116.23	43.49	233
2	112.62	43.87	230
5	116.96	44.17	228

At $\lambda = 0.1$, pending p95 falls from 256 to 244 s and matched p95 from 123 to 119 s. Serving older players raises the completed mean wait; all-arrival queue residence stays 109.912 s. The reward redistributes service without removing the role bottleneck.

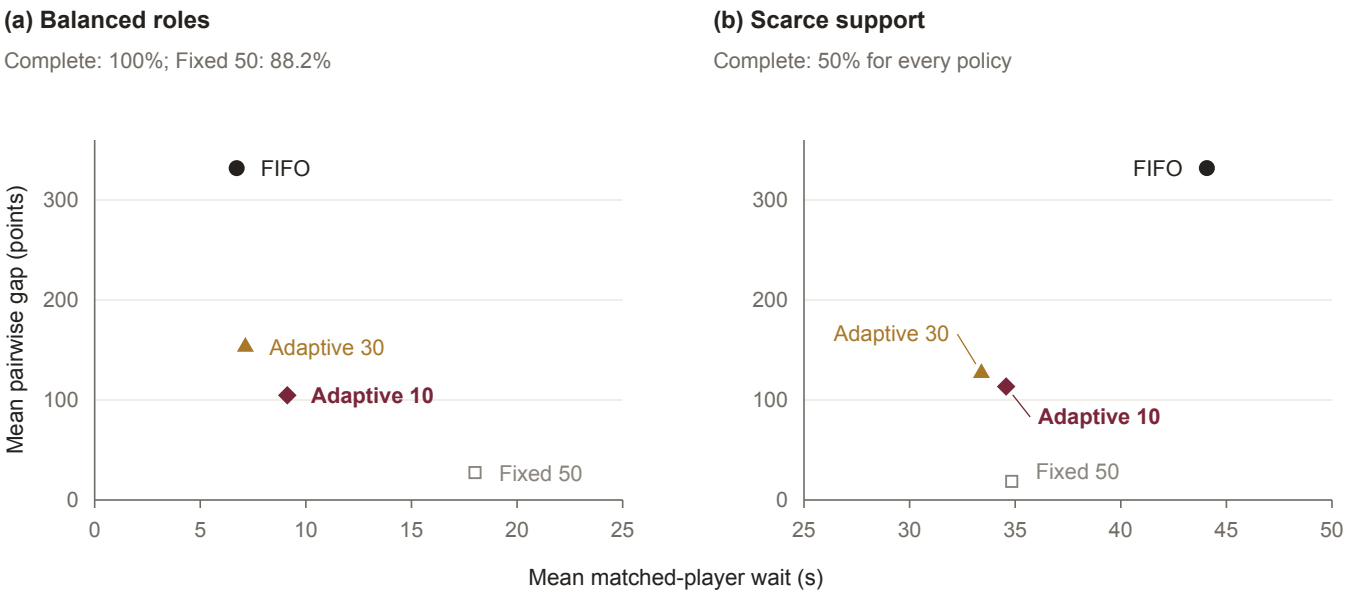


Figure 2. Each point pools five runs; uncertainty is not shown. The gap axis is shared, but wait-axis ranges differ. Completion is given under each title. Matched-only waits condition on service; Figure 3 also counts pending requests.

5. Experimental Setup

All policies receive the same stream for each seed and scenario. A run contains 1,000 arrivals: five players per second for $t = 0,...,199$. At each tick, the five arrivals are enqueued before matching. Matching then continues through $H = 319$, providing 120 additional drain ticks.

Scores are uniform integers in [1000, 2000]. The balanced scenario contains exactly 200 players per role. The support-scarce scenario contains 225 players in each other role and 100 supports. A Fisher-Yates shuffle randomizes arrival roles; SplitMix64 with rejection sampling generates the bounded integers. Seeds 1,...,5 produce five runs per scenario.

5.1. Compared Policies

FIFO takes the oldest player from each role whenever all five roles are available, without score filtering. *Fixed 50* uses the same anchor scan and exact optimizer as the proposed policy, but keeps the band at 50. *Adaptive 10* and *Adaptive 30* start at 50 and widen by 10 or 30 points/s, with a cap of 10^6.

Figure 2 plots pooled waiting time and score gap, with completion rates stated above each panel. Table 3 reports these pooled metrics; its p95 is the nearest-rank percentile of matched-player waits. Pending counts and ages are measured at the common cutoff. The five runs describe synthetic workloads rather than a sample of commercial-game traffic.

6. Results

6.1. Balanced Roles

Adaptive 10 matches every arrival and lowers the pooled mean gap from 331.86 to 104.69 points, a 68.5% reduction relative to FIFO. Its mean wait is 9.12 s instead of 6.72 s, and p95 rises from 18 to 30 s. Thus the gain in score similarity is accompanied by both a higher mean wait and a longer tail.

Increasing the growth rate to 30 moves the policy toward FIFO: mean wait falls to 7.14 s, while gap rises to 153.17 points. Both adaptive settings finish all players by the cutoff. Fixed 50 gives the smallest gap, 27.44 points, but only 88.2% completion, leaving 590 players pending across the five runs.

Changing the growth rate changes eligibility, not the exact optimizer. A fixed band can leave globally available roles locally incompatible; widening resolves such score-related blocks.

The separate age-reward sweep also matches all balanced arrivals. Mean wait ranges from 9.01 to 9.17 s and gap from 101.32 to 104.69 points across the seven weights; changes are small and not monotone.

6.2. Scarce Support

Only 100 supports arrive in each scarce run. The role constraint therefore permits at most 100 teams, or 50% of the 1,000 players. Every tested policy reaches this bound. Unlike the balanced case, Fixed 50 sacrifices no completion rate here and yields the smallest gap, 18.60 points; adaptive widening is not uniformly preferable.

Table 3. Pooled results for five seeds and 5,000 arrivals per policy in each scenario. Mean wait and p95 refer only to matched players. W_H includes observed residence time for every arrival through $H = 319$ s (Eq. (8)). Gap is the mean of the ten pairwise differences per team.

Roles	Policy	Match (%)	Mean wait (s)	p95 (s)	Gap (pts)	W_H (s)	Pending
Balanced	FIFO	100.0	6.72	18	331.86	6.72	0
	Fixed 50	88.2	18.01	65	27.44	33.61	590
	Adaptive 10	100.0	9.12	30	104.69	9.12	0
	Adaptive 30	100.0	7.14	24	153.17	7.14	0
Scarce	FIFO	50.0	44.08	107	331.92	109.71	2,500
	Fixed 50	50.0	34.84	122	18.60	110.69	2,500
	Adaptive 10	50.0	34.57	123	113.48	109.91	2,500
	Adaptive 30	50.0	33.40	122	126.91	109.77	2,500

6.3. Accounting for Unmatched Players

To keep pending requests in the evaluation, let d_i be player i's match time, or infinity if unmatched. Define the observed queue residence time

$$W_H = \frac{1}{N} \sum_{i=1}^{N} [\min(d_i, H) - a_i]. \quad (8)$$

This quantity counts each completed wait and each pending player's current age. It is truncated at the cutoff, not an estimate of eventual completed waiting time.

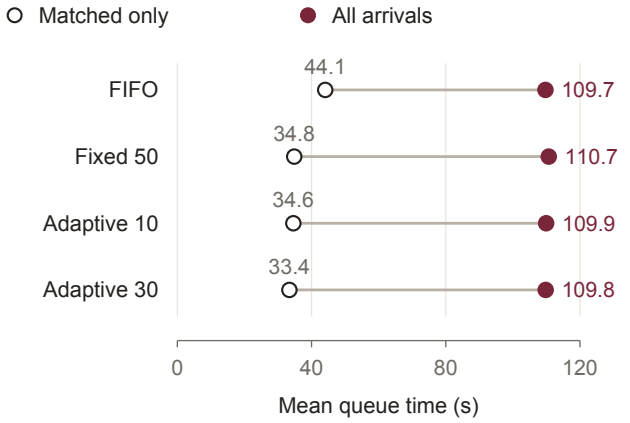


Figure 3. Scarce support. Open circles average completed waits; filled circles show W_H, the observed queue residence of every arrival through the cutoff. Values pool five runs. Each line compares different populations, not additive parts of a wait.

In the scarce setting, FIFO has $W_H = 109.71$ s and Adaptive 10 has 109.91 s, despite the latter's lower matched-only mean. Their oldest pending requests are 238 and 273 s old, respectively (maximum over seeds). Each policy leaves 625 players in each non-support role and no support pending.

7. Discussion and Conclusion

Widening is useful when score compatibility, rather than role supply, prevents progress. The balanced experiment illustrates that case. The scarce experiment illustrates the opposite: enough score-compatible combinations exist to use every support, and no policy can exceed the role-supply limit.

Age priority gives no minimum-wait guarantee: an infeasible player can remain behind, and newer players may join a team. Missing roles can cause unbounded waiting. Parties, latency, and individual score limits would require additional constraints.

The age reward adds a controlled preference for older teammates. It can change later candidate pools, so Table 2 need not be monotone in gap. The capped reward provides no starvation guarantee. A useful next step is to distinguish ticks blocked by missing roles from those blocked only by score compatibility.

A small exact optimizer makes the online policy easy to test. In balanced streams, growth 10 yields closer teams for 2.40 s of extra mean wait. Choosing a band and age reward should depend on role supply and acceptable queue tails.

References

Herbrich, R., Minka, T., and Graepel, T. TrueSkill: A Bayesian skill rating system. In *Advances in Neural Information Processing Systems*, 19:569-576, 2006. Author manuscript.

ICML. *Submission and Formatting Instructions for ICML 2026*, 2026. Official example paper. Used as a typography and figure-format reference.

Course assignment. *Implementing a game matching algorithm.* Problem statement supplied as an image. The report adopts its five-role requirement and two optimization objectives.

Appendix A. Implementation and Reproducibility

Table 4. Local microbenchmarks, reported as median elapsed time over seven runs. Both versions use C++17 and Apple Clang 21 with -O2 on the same ARM64 macOS environment. Queue construction is excluded. The first two workloads allow unrestricted score bands.

Workload	Original (ms)	Revised (ms)	What is measured
500 players; equal scores	45.464	1.919	One call forming 100 teams
500 players; mixed scores	15.867	3.179	One call forming 100 teams
5,000 players; missing role	9.007	0.038	100 ticks; no teams formed

A.1. Implementation Choices

The reference implementation is single-threaded. Joining and cancellation scan the vector for a player ID and take $O(n)$ time. Matching removes all five selected records before the next search. The CLI rejects extra tokens before mutating queue state, and request/match times must be nonnegative and nondecreasing.

Each DP state now stores five fixed player slots and a sorted age/ID key. Inserting a candidate updates that key directly, avoiding temporary vectors and repeated sorting during tie comparisons. A preliminary role-mask scan returns before building a score index when a required role is globally absent. These changes preserve the selection rule and improve constants; they do not change the general batch bound.

Table 4 compares the preserved original header with the revised implementation. The speedups depend on workload and compiler behavior. They should not be interpreted as server throughput, and the benchmark excludes queue construction, which still uses linear ID checks.

A.2. Verification

The independent oracle enumerates all role combinations for the first feasible anchor, sums the ten absolute differences directly, and sorts the final age/ID key. It does not use Eq. (4). Across 1,000 seeded queues, all 1,437 selected teams match the oracle exactly.

Assertions also check role uniqueness, anchor inclusion, band eligibility, unchanged player records, no reuse, and absence of a feasible team after a matching batch. Targeted cases cover missing roles, equal timestamps, fractional growth, caps, score extremes, cancellation/rejoining, invalid policies, and timestamps near the integer limit.

Five malformed CLI commands are rejected without side effects. The test suite also passes AddressSanitizer and UndefinedBehaviorSanitizer without diagnostics.

The extension passes 3,822 exhaustive checks over 2,100 queues, including the gap bound. At $\lambda = 0$, IDs and times match the baseline on 300 queues and ten streams. Its sanitizer checks also pass.

A.3. Reproducing the Results

The project builds with a C++17 compiler and `make`. Python 3 is used only for the CLI regression and report-generation utilities. From the project directory:

```
make
make waiting-experiment
make test
make sanitize
make experiment
make benchmark-compare
```

The simulation writes `experiment_results.json`, including every seed and pooled metrics. The timing command writes both summaries under `results/`. The original matching header is retained under `reference/`. The separate age-reward prototype and full sweep instructions are under `research/`.

A.4. A Small End-to-End Example

Enter the following commands after `make run`. Roles are abbreviated as top, jun, mid, adc, and sup.

```
join 1 top 1500 0
join 2 jun 1510 1
join 3 mid 1490 2
join 4 adc 1520 3
join 5 sup 1480 4
match 4
quit
```

At time 4, player 1 is the anchor and its band is $50 + 10 \times 4 = 90$ points. All roles lie within [1410, 1590]. The waits are 4, 3, 2, 1, 0 s, so the mean is 2 s. The sorted scores are 1480, 1490, 1500, 1510, 1520; Eq. (4) gives a pairwise sum of 200 and a mean gap of 20.

A.5. Reporting Conventions

Figures 2 and 3 and Table 3 pool observations over five runs. Per-seed data and run-level standard deviations are supplied separately; no confidence interval or significance test is claimed. Baseline measurements were saved on 19 September 2026; the age-reward sweep was added on 11 October 2026. The complete per-seed sweep is included with the submission.